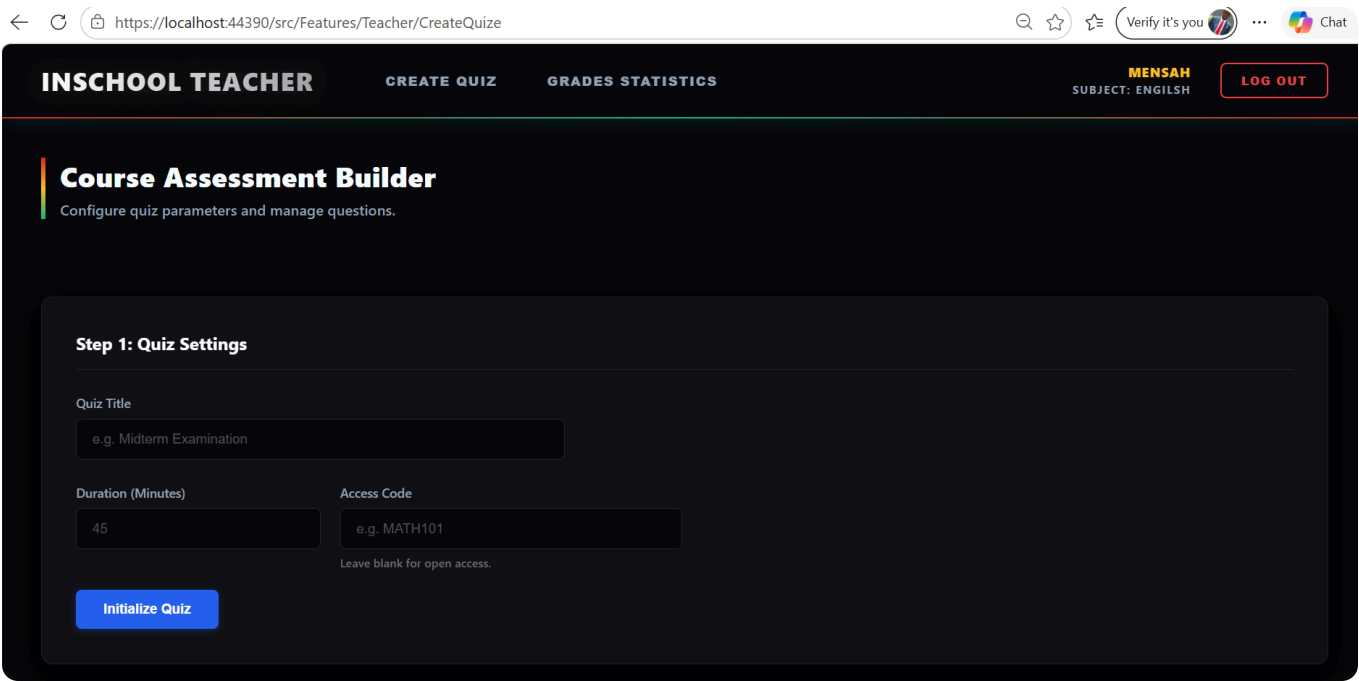


Teacher Module - Quick Guide



What Teachers Do With Quizzes



Activity 1: Creating a Quiz (Assessment Builder)

←↻🔍https://localhost:44390/src/Features/Teacher/CreateQuiz

🔍☆🔖Verify it's you...🗺️Chat

INSCHOOL TEACHER

CREATE QUIZ

GRADES STATISTICS

MENSAH
SUBJECT: ENGLISH

LOG OUT

Course Assessment Builder

Configure quiz parameters and manage questions.

Step 1: Quiz Settings

Quiz Title

e.g. Midterm Examination

Duration (Minutes)

45

Access Code

e.g. MATH101

Leave blank for open access.

Initialize Quiz

What Happens

A teacher creates an exam with multiple-choice questions and settings.

Step-by-Step

1. **Select Course**: Teacher picks which course this quiz is for
2. **Set Quiz Details**:
 - Quiz name: "Midterm Exam"
 - Time limit: 60 minutes
 - Access code: "MIDTERM2024"
3. **Add Questions**: Teacher adds multiple-choice questions one by one
 - Question text: "What is 2+2?"
 - Option A: "3"
 - Option B: "4" ← **Correct Answer**
 - Option C: "5"
 - Option D: "6"
4. **Save**: System saves quiz to database

Why It's Built This Way

Hidden Tracking

When a teacher creates a quiz, the system assigns it an **ID number** (like a social security number for the quiz).

The system **silently remembers** this ID as the teacher adds questions. This way:

- Every question gets linked to the right quiz
- Questions don't get lost between clicks
- No manual ID management needed

Correct Answer Selection

Instead of letting teachers **type** the correct answer (which could lead to typos), they **select from a dropdown menu**:

- Choice A
- Choice B
- Choice C
- Choice D

Why? If a teacher types "Apple" by mistake instead of "A", the auto-grader breaks. The dropdown prevents this.

Security

All question text is validated before saving to prevent:

- Accidental data corruption
- Security issues from special characters

Activity 2: Reviewing Results & Closing Quiz

What Happens

After students take the quiz, the teacher can:

- 1. See student scores
- 2. Download grades to Excel
- 3. Lock/close the quiz



Part A: View Quiz Results

What Teacher Sees

A table showing:

Student Name	Score	Grade
Alice Johnson	85	B
Bob Smith	92	A
Catherine Lee	78	C

Color-Coded Grades

- **Green** = Excellent (A)
- **Yellow** = Good (B-C)
- **Red** = Needs Improvement (D-F)

Teachers instantly see who did well and who needs help.

Why Automated Colors?

The system **automatically** colors the grades based on the score. Teacher doesn't have to do it manually. It's built in.



Part B: Download Grades to Excel

What It Does

Teacher clicks "Export to Excel" and a file downloads to their computer.

They can then:

- Print it
- Email it to parents
- Keep records
- Analyze trends

Why This Matters

Teachers can work with the data offline in Excel without needing to stay in the system.



Part C: Close the Quiz

What Happens

When the teacher clicks "Close Quiz":

- 1. Quiz is marked as CLOSED (no more students can take it)
- 2. Any student still taking it → Auto-submitted with their current score
- 3. Quiz is locked permanently
- 4. Database is updated in one atomic operation

Why Lock the Quiz?

If a teacher forgets to close a quiz:

- Students could keep taking it
- Grades could change randomly
- Academic integrity is compromised

The lock button prevents this.

Why Auto-Submit Students?

If a student is halfway through when the quiz closes:

- They get a score based on what they answered
- Their answers aren't lost
- The system has a complete record



Key Features Explained

Feature	What It Does	Why It Matters
Hidden Quiz ID	System remembers which quiz you're adding to	Questions go to the right quiz
Dropdown for Correct Answer	Forces selection of A, B, C, or D	Prevents typos that break auto-grader

Feature	What It Does	Why It Matters
Instant Result Colors	Green for A, Red for F	Teacher sees performance at a glance
Auto-Selected Dropdown	Selecting a quiz instantly loads results	No "Search" button needed
Export to Excel	Download grades for offline use	Can work with grades anywhere
Atomic Quiz Close	All-or-nothing operation	No partial failures, no data corruption
Auto-Submit Students	Incomplete quizzes auto-finished	No one trapped in broken state



Simple Example: Creating a Quiz

Teacher: "I want to create a quiz for Biology class"

System: "OK, let me give this quiz an ID number"

System: "ID = 1001"

Teacher: "I'm adding Question 1: What is photosynthesis?"

System: "Linking Question 1 to Quiz 1001"

Teacher: "I'm adding Question 2: How many chromosomes..."

System: "Linking Question 2 to Quiz 1001"

Teacher: "Done!"

System: "Quiz 1001 saved with 2 questions"



Simple Example: Closing a Quiz

Teacher: "Close the quiz"

System: "Starting close process..."

System: "Step 1: Mark quiz as CLOSED"

System: "Step 2: Find Alice still taking quiz"

System: "Step 3: Auto-submit Alice with 65% (based on answers so far)"

System: "Step 4: Find Bob still taking quiz"

System: "Step 5: Auto-submit Bob with 78%"

System: "Step 6: Lock quiz permanently"

System: "Complete! Quiz closed, all students submitted."



What If Something Goes Wrong?

Quiz Won't Save

Reason: Question text might be too long or have special characters **Solution:** Check question length, try again

Grades Wrong Color

Reason: System calculates grade wrong or display hasn't refreshed **Solution:** Refresh page or check individual answers

Can't Close Quiz

Reason: Server connection lost mid-process **Solution:** System rolls back and tries again (nothing corrupted)



The Presenter's Simple Pitch

// "The Teacher Module lets educators create and manage quizzes efficiently.

Creating: Teachers pick a course, set quiz parameters, and add multiple-choice questions one by one. The system automatically tracks which questions belong to which quiz, preventing data mix-ups.

Reviewing: Teachers see student performance with color-coded grades. Green for excellence, red for needs improvement. They can instantly identify who needs help.

Exporting: Teachers download grades to Excel for offline use, printing, or email.

Closing: When a teacher closes a quiz, the system locks it, auto-submits any students still taking it, and updates everything in one atomic operation. No partial failures. No data corruption. Academic integrity guaranteed."



Quick Summary

Creating Quizzes: Teachers add course, quiz settings, and questions. System automatically tracks everything.

Viewing Results: Instant dashboard with color-coded grades and student performance.

Exporting Grades: One click downloads everything to Excel.

Closing Exams: One-button close that safely locks quiz and finishes any incomplete attempts.

Why It Works: Built-in tracking, automatic validation, color-coded feedback, and safe shutdown procedures.



FAQ

Q: What if I accidentally add a question to the wrong quiz? A: You can edit the quiz and delete the wrong question, then re-add it to the correct quiz.

Q: Can students retake the quiz after I close it? A: No. Once closed, it's locked. They can review their answers but not retake it.

Q: What if the internet cuts out while I'm closing a quiz? A: The system rolls back. Either everything saves or nothing saves. No partial/broken states.

Q: Can I edit a quiz after creating it? A: Yes. You can add, edit, or delete questions before the quiz is closed.

Q: What happens to students' answers after I close the quiz? A: They're saved and locked. Students see their grades but can't change answers.

```
CREATE TABLE IF NOT EXISTS users (  
  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    username VARCHAR(50) NOT NULL UNIQUE,  
  
    password_hash VARCHAR(255) NOT NULL,  
  
    role ENUM(  
        'ADMIN',  
        'STUDENT',  
        'SECURITY'  
    ) NOT NULL,  
  
    full_name VARCHAR(150) NOT NULL,  
  
    campus_id INT NULL,  
  
7   phone_number VARCHAR(20) NULL,  
  
    profile_photo_path VARCHAR(255) NULL,  
  
    first_login_completed BOOLEAN NOT NULL DEFAULT FALSE,  
  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
        ON UPDATE CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS campuses (
```

```
    campus_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    campus_name VARCHAR(100) NOT NULL UNIQUE,
```

```
    campus_code VARCHAR(20) NOT NULL UNIQUE,
```

```
    campus_description VARCHAR(255) NULL,
```

```
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
    ON UPDATE CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS incident_categories (
```

```
    category_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    category_name VARCHAR(100) NOT NULL UNIQUE,
```

```
    category_description VARCHAR(500) NULL,
```

```
    severity_default ENUM(
```

```
        'LOW',
```

```
        'MEDIUM',
```

```
        'HIGH',
```

```
        'CRITICAL'
```

```
) NOT NULL DEFAULT 'MEDIUM',
```

```
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS incidents (

    incident_id INT AUTO_INCREMENT PRIMARY KEY,

    incident_reference VARCHAR(30) NOT NULL UNIQUE,

    reported_by_user_id INT NOT NULL,

    campus_id INT NOT NULL,

    category_id INT NOT NULL,

    incident_title VARCHAR(200) NOT NULL,

    incident_description TEXT NOT NULL,

    incident_location VARCHAR(255) NOT NULL,

    priority_level ENUM(
        'LOW',
        'MEDIUM',
        'HIGH',
        'CRITICAL'
    ) NOT NULL DEFAULT 'MEDIUM',

    evidence_image_path VARCHAR(255) NULL,

    current_status ENUM(
        'PENDING',
        'ASSIGNED',
        'IN_PROGRESS',
        'RESOLVED',
        'CLOSED',
        'REJECTED'
    ) NOT NULL DEFAULT 'PENDING',

    reported_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
closed_at DATETIME NULL,  
  
CONSTRAINT fk_incident_reporter  
    FOREIGN KEY (reported_by_user_id)  
    REFERENCES users(user_id),  
  
CONSTRAINT fk_incident_campus  
    FOREIGN KEY (campus_id)  
    REFERENCES campuses(campus_id),  
  
CONSTRAINT fk_incident_category  
    FOREIGN KEY (category_id)  
    REFERENCES incident_categories(category_id)  
  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS incident_assignments (  
  
    assignment_id INT AUTO_INCREMENT PRIMARY KEY,  
  
    incident_id INT NOT NULL,  
  
    assigned_by_user_id INT NOT NULL,  
  
    assigned_to_user_id INT NOT NULL,  
  
    assignment_notes TEXT NULL,  
  
    assignment_status ENUM(  
        'ACTIVE',  
        'REASSIGNED',  
        'COMPLETED',  
        'CANCELLED'  
    ) NOT NULL DEFAULT 'ACTIVE',  
  
    assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    completed_at DATETIME NULL,  
  
    CONSTRAINT fk_assignment_incident  
        FOREIGN KEY (incident_id)  
        REFERENCES incidents(incident_id),
```

```
CONSTRAINT fk_assignment_admin
    FOREIGN KEY (assigned_by_user_id)
    REFERENCES users(user_id),

CONSTRAINT fk_assignment_security
    FOREIGN KEY (assigned_to_user_id)
    REFERENCES users(user_id)

) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS incident_responses (

    response_id INT AUTO_INCREMENT PRIMARY KEY,

    incident_id INT NOT NULL,

    responded_by_user_id INT NOT NULL,

    response_type ENUM(
        'INVESTIGATION_UPDATE',
        'ACTION_TAKEN',
        'FINAL_REPORT'
    ) NOT NULL,

    response_details TEXT NOT NULL,

    evidence_image_path VARCHAR(255) NULL,

    response_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_response_incident
        FOREIGN KEY (incident_id)
        REFERENCES incidents(incident_id),
```

```
CONSTRAINT fk_response_security
    FOREIGN KEY (responded_by_user_id)
    REFERENCES users(user_id)
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS incident_status_history (

    status_history_id INT AUTO_INCREMENT PRIMARY KEY,

    incident_id INT NOT NULL,

    changed_by_user_id INT NOT NULL,

    previous_status ENUM(
        'PENDING',
        'ASSIGNED',
        'IN_PROGRESS',
        'RESOLVED',
        'CLOSED',
        'REJECTED'
    ) NOT NULL,

    new_status ENUM(
        'PENDING',
        'ASSIGNED',
        'IN_PROGRESS',
        'RESOLVED',
        'CLOSED',
        'REJECTED'
    ) NOT NULL,

    change_reason VARCHAR(255) NULL,

    changed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_status_incident
        FOREIGN KEY (incident_id)
        REFERENCES incidents(incident_id),

    CONSTRAINT fk_status_user
```

```
FOREIGN KEY (changed_by_user_id)  
REFERENCES users(user_id)
```

```
) ENGINE=InnoDB;
```